

wsl中的docker不同安装形式

在 WSL 环境下使用 Docker 时，主要有两种方式：Docker Desktop（集成 WSL 后端）和 直接在 WSL 发行版中手动安装 Docker。以下是它们的区别及安装位置详解：

1. Docker Desktop集成 WSL 后端

1.1 特点

- **安装方式：**通过 Windows 安装程序安装 Docker Desktop，并启用 WSL 2 后端支持。
- **运行机制：**
 - Docker Desktop 会在后台启动一个轻量级的 Linux 虚拟机（基于 WSL 2），名为 `docker-desktop` 和 `docker-desktop-data`。
 - Docker 守护进程（`dockerd`）运行在 `docker-desktop` 虚拟机中，而非直接运行在用户安装的 WSL 发行版中。
 - 用户可以在任意 WSL 发行版中直接使用 `docker` 命令，Docker Desktop 会自动将命令转发到后台守护进程。
- **资源管理：**
 - Docker Desktop 自动管理资源（CPU、内存、存储），与 WSL 2 共享资源。
 - 镜像和容器数据存储在专用的 WSL 发行版 `docker-desktop-data` 中。
- **功能支持：**
 - 提供图形界面和完整的 Docker 生态工具（如 Docker Compose、Kubernetes）。
 - 支持与 Windows 文件系统无缝集成（通过 `/mnt/c` 访问 Windows 文件）。

1.2 安装位置

- **主程序：**
 - Windows 路径：`C:\Program Files\Docker\Docker`
- **镜像与容器数据：**
 - 存储在 WSL 发行版 `docker-desktop-data` 中，路径为 `\\ws1$\docker-desktop-data\version-pack-data\community\docker`（通过 Windows 资源管理器访问）。
 - 在 WSL 发行版内部，数据位于 `/var/lib/docker`。

2. 直接在 WSL 发行版中安装 Docker

2.1 特点

- **安装方式：**在 WSL 的 Linux 发行版（如 Ubuntu）中，手动通过 `apt` 安装 Docker 引擎（`docker.io` 或 `docker-ce`）。
- **运行机制：**
 - Docker 守护进程（`dockerd`）直接运行在 WSL 发行版中。
 - 需要手动启动 Docker 服务（`sudo service docker start`）。
- **资源管理：**
 - 容器和镜像存储在 WSL 发行版的文件系统中（如 `/var/lib/docker`）。

- 资源 (CPU/内存) 受限于 WSL 发行版的配置 (通过 `.wslconfig` 调整)。
- 功能支持:
 - 仅支持命令行操作, 无图形界面。
 - 需要手动配置 Docker 客户端与守护进程的通信 (通常通过 Unix 套接字 `unix:///var/run/docker.sock`)。

2.2 安装位置

- 主程序:
 - WSL 发行版中的 Linux 路径: `/usr/bin/docker` (客户端) 和 `/usr/bin/dockerd` (守护进程)。
- 镜像与容器数据:
 - 存储在 WSL 发行版内: `/var/lib/docker`。

3. 关键区别总结

对比项	Docker Desktop (WSL 后端)	直接在 WSL 中安装的 Docker
安装复杂度	一键安装, 自动配置	手动安装, 需配置服务和权限
运行环境	后台 WSL 虚拟机 (<code>docker-desktop</code>)	当前 WSL 发行版
资源隔离	专用虚拟机隔离资源	共享 WSL 发行版资源
图形界面	支持 Docker Dashboard	仅命令行
数据存储位置	<code>\\wsl\$\\docker-desktop-data</code> (Windows 可见)	WSL 发行版的 <code>/var/lib/docker</code>
Kubernetes 支持	内置支持	需手动部署
性能	优化后的 WSL 2 集成	依赖 WSL 发行版配置

Table 1

3.1 如何选择?

- 推荐 Docker Desktop:
适合需要图形界面、Kubernetes 支持或希望简化管理的用户, 尤其是开发跨平台应用。
- 推荐手动安装:
适合熟悉 Linux 管理、希望减少资源占用 (关闭 Docker Desktop 后台进程) 或需要完全控制 Docker 环境的用户。

3.2 附加提示

- 若同时使用两种方式，注意避免端口冲突（如 2375/2376）。
- 通过 `docker context` 命令可切换 Docker 客户端连接的目标（如本地 WSL 或远程服务器）。

wsl与docker desktop的关系

在 Windows 系统上，Docker Desktop 依赖于wsl来运行 Linux 内核，从而支持 Docker 引擎的运行

1. 关系说明

- 1. **WSL 提供基础环境**：WSL 是 Windows 上运行 Linux 二进制可执行文件（ELF 格式）的兼容层，它包含了一个轻量级的 Linux 内核以及相关的系统工具。在 Docker Desktop 的运行过程中，WSL 负责提供底层的 Linux 环境，使得 Docker 引擎能够在 Windows 系统上正常运行。
- 2. **Docker Desktop 集成和管理**：Docker Desktop 基于 WSL 构建，它集成了 Docker 引擎、Docker CLI 客户端以及 Docker Compose 等工具，为用户提供了一个图形化和命令行相结合的界面，方便用户管理 Docker 容器、镜像和网络等资源。用户可以通过 Docker Desktop 的图形界面轻松创建、启动、停止和删除容器，也可以使用命令行进行更高级的操作。
- 3. **数据交互**：Docker Desktop 利用 WSL 的文件系统共享功能，实现了 Windows 文件系统和 Docker 容器内文件系统之间的数据交互。用户可以将 Windows 系统上的文件和目录挂载到 Docker 容器中，方便在容器内访问和处理这些数据

2. 示意图

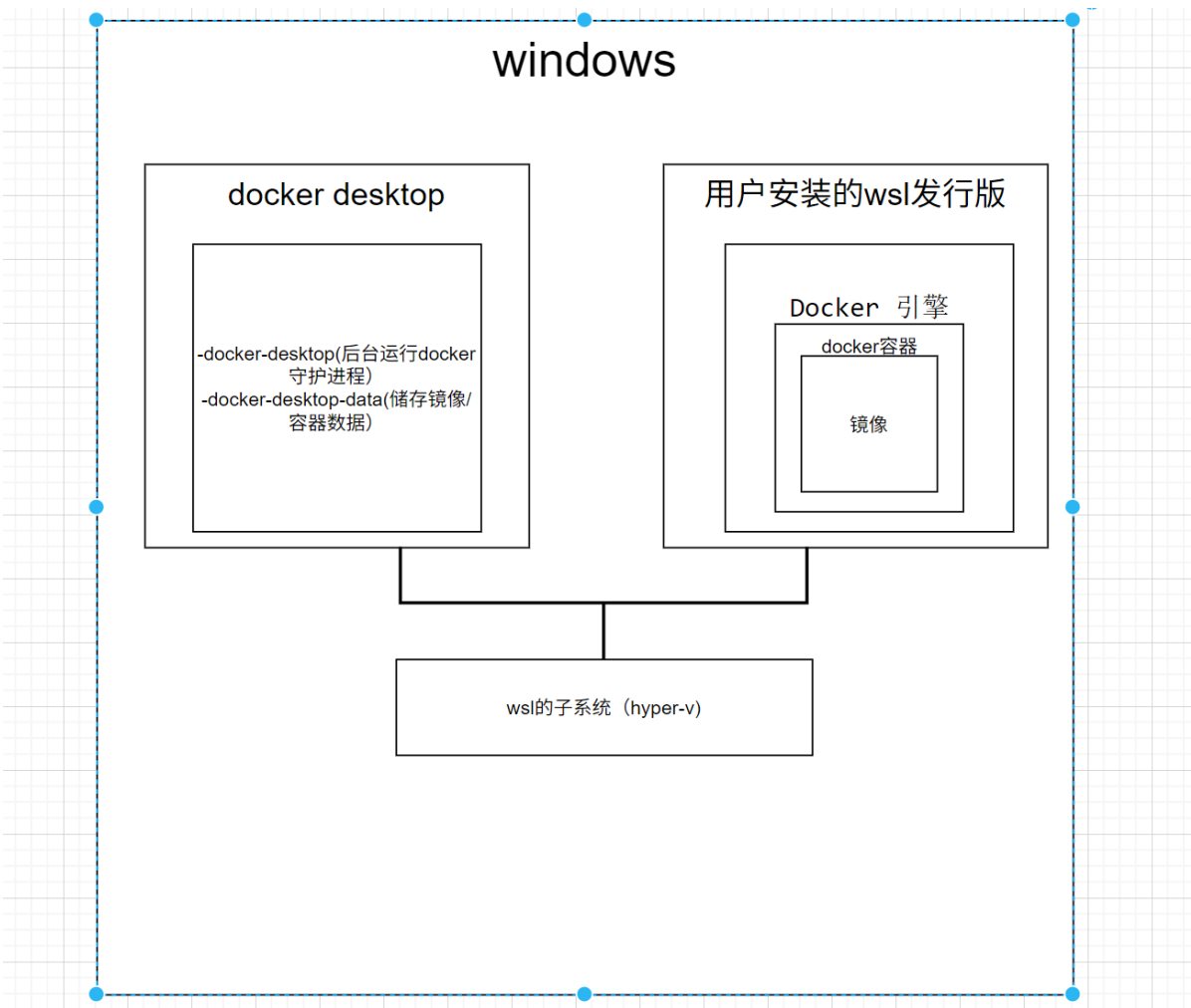


Figure 1

2.1 关键关系解析

1. Docker Desktop 的后台 WSL 虚拟机

- Docker Desktop 默认创建两个专用 WSL 发行版：
 - `docker-desktop`：运行 Docker 守护进程（`dockerd`）。
 - `docker-desktop-data`：存储镜像、容器等数据（位于 `/var/lib/docker`）。
- 这些虚拟机由 Docker Desktop 自动管理，用户无需直接操作。

2. 用户安装的 WSL 发行版

- 用户自行安装的 WSL 发行版（如 Ubuntu）中，可以直接使用 `docker` 命令。
- Docker CLI 默认通过 Unix 套接字（`/var/run/docker.sock`）或 TCP 连接到 `docker-desktop` 虚拟机中的守护进程，**无需手动配置**。

3. WSL 2 子系统

- 所有 WSL 发行版（包括 Docker Desktop 的虚拟机和用户安装的发行版）均运行在 WSL 2 的虚拟化层上，依赖 Hyper-V 提供 Linux 内核和资源隔离。

4. 文件系统互通

- WSL 发行版可直接访问 Windows 文件系统（通过 `/mnt/c` 等路径）。
- Docker 容器通过 Volume 挂载实现与 Windows/WSL 文件系统的交互。

5. 网络互通

- Docker 容器网络与 WSL 发行版共享同一网络栈，可通过 `localhost` 直接访问容器端口（无需配置 `-p 127.0.0.1:8080:80`）

2.2 协作流程

1. 用户在 WSL 发行版（如 Ubuntu）中执行 `docker run` 命令。
2. Docker CLI 将命令转发到 `docker-desktop` 虚拟机中的守护进程。
3. 守护进程创建容器，容器进程运行在 `docker-desktop` 虚拟机的隔离环境中。
4. 容器数据存储在 `docker-desktop-data` 虚拟机中，与用户 WSL 发行版分离。
5. 用户可通过 Windows 资源管理器访问 `\\ws1\docker-desktop-data` 查看镜像和容器数据

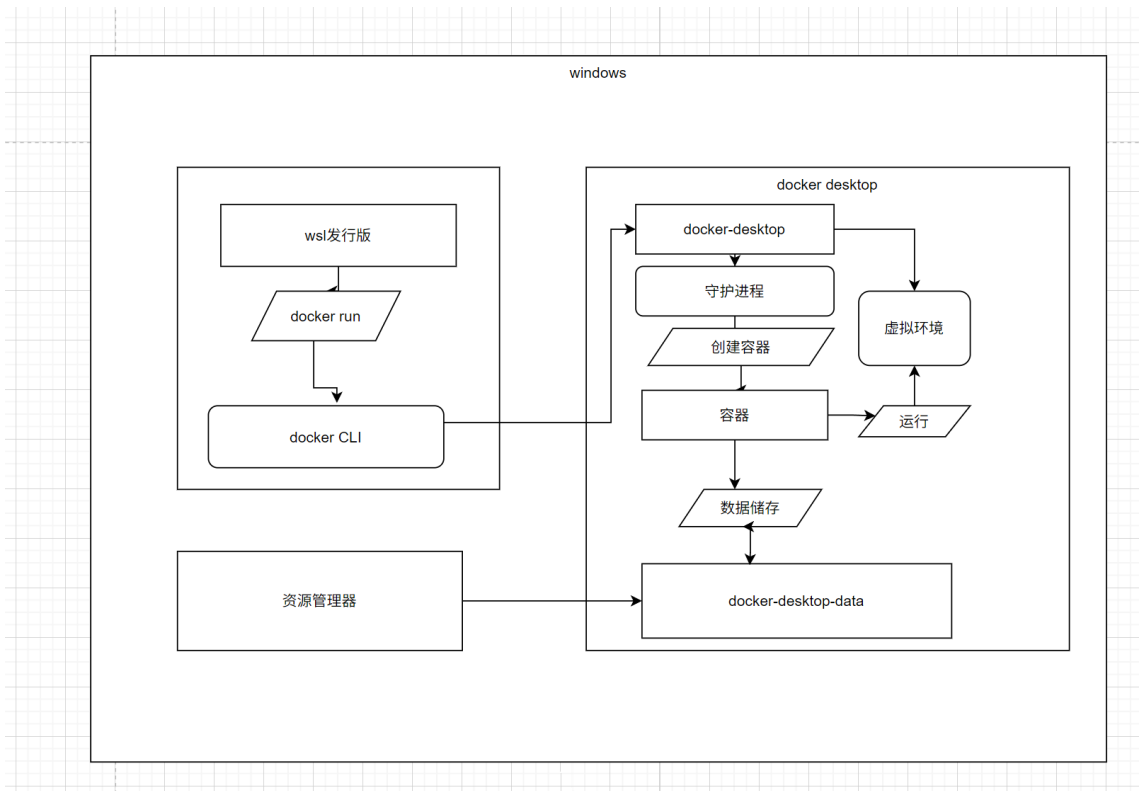


Figure 2

2.3 对比手动在 WSL 中安装 Docker

若手动在 WSL 发行版中安装 Docker（不依赖 Docker Desktop）：

- Docker 守护进程直接运行在用户 WSL 发行版中，与 Docker Desktop 的虚拟机无关。
- 镜像和容器数据存储在用户 WSL 发行版的 `/var/lib/docker` 目录下。
- 需手动管理服务（如 `sudo service docker start`），且无图形界面支持

2.4 总结

Docker Desktop 通过 WSL 2 虚拟机实现 Docker 环境与 Windows 的深度集成，而用户安装的 WSL 发行版仅作为客户端使用。这种设计既保证了性能，又简化了跨平台开发流程